

# Graphs: Strongly Connected Components, Bridges, and Articulation Points

A Problem-Solving Companion to Striver's A2Z and SDE Sheets

Maruri Saikiran

June 30, 2026



# Contents

|          |                                                                                |          |
|----------|--------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Graphs: Strongly Connected Components, Bridges, and Articulation Points</b> | <b>1</b> |
| 1.1      | Discovery Time and Low-Link Value . . . . .                                    | 1        |
| 1.2      | Strongly Connected Components (Kosaraju's Algorithm) . . . . .                 | 2        |
| 1.3      | Bridges in a Graph . . . . .                                                   | 4        |
| 1.4      | Articulation Points . . . . .                                                  | 6        |



# Chapter 1

## Graphs: Strongly Connected Components, Bridges, and Articulation Points

*This closing graph chapter asks about structural weak points and tightly-knit clusters: which groups of nodes can all reach each other (strongly connected components), and which single edges or nodes – if removed – would break the graph apart (bridges and articulation points)? Both questions are answered by one DFS, carrying a little extra bookkeeping about when each node was discovered.*

### 1.1 Discovery Time and Low-Link Value

Sections 1.3 and 1.4 both rely on two numbers computed during a single DFS pass:

#### The Two Numbers Every Node Tracks

- $tin[node]$ : the **discovery time** – the order in which DFS first visits this node (a simple counter, incremented on each new visit).
- $low[node]$ : the **lowest discovery time reachable** from this node’s DFS subtree, using any combination of tree edges (down to children) and **back edges** (up to an already-visited ancestor) – but **never** the single tree edge back to the immediate parent, since that edge doesn’t represent genuine alternate connectivity.

A node’s  $low$  value answers: “ignoring the edge I arrived through, how far back up the DFS tree can I still reach?” – and that single question, asked at every node, is what both bridges and articulation points are built from.

#### What This Chapter Covers

- **Strongly Connected Components (Section 1.2)**: a different technique entirely (Kosaraju’s algorithm, using finish-time ordering and a graph transpose) for the directed-graph analogue of “which nodes can all reach each other.”
- **Bridges in a Graph (Section 1.3)**: an edge  $u \rightarrow v$  (tree edge) is a bridge exactly when  $v$ ’s subtree has **no** back edge reaching  $u$  or higher – i.e.  $low[v] > tin[u]$ .
- **Articulation Points (Section 1.4)**: a near-identical condition,  $low[v] \geq tin[u]$  (note:  $\geq$ , not  $>$ ), identifying **nodes** whose removal disconnects the graph, with one special case for the DFS root.

## 1.2 Strongly Connected Components (Kosaraju’s Algorithm)

**Problem Statement.** Given a directed graph, find its **Strongly Connected Components** (SCCs) – maximal groups of nodes where every node can reach every other node in the same group via directed edges.

### Pattern Recognition

“Group nodes that can all reach each other” is solved by Kosaraju’s algorithm: run DFS once to compute a finish-time order (exactly the Graph Topological Sort chapter’s finish-time idea), then run DFS **again** on the graph with every edge **reversed**, processing nodes in **decreasing** finish-time order – each DFS launched in this second pass visits exactly one complete SCC.

### Why reversing edges and using finish-time order works

Reversing every edge preserves SCCs exactly (if  $u$  and  $v$  can reach each other in the original graph, they can still reach each other with every edge flipped) but **destroys** inter-SCC reachability in a specific, useful direction: collapsing each SCC into a single “super-node” produces a DAG (the **condensation** of the graph), and processing nodes in decreasing finish-time order from the *first* DFS guarantees the first unvisited node encountered in the second pass always belongs to a **source** SCC (no incoming edges from any other not-yet-visited SCC) of the *original* condensation – which is a **sink** SCC once edges are reversed. Starting DFS there in the reversed graph cannot leak into any other SCC, since reversed edges now only point *into* this SCC from elsewhere, not out of it.

### Algorithm

1. Run DFS over the original graph from every unvisited node, pushing each node onto a stack when it finishes (identical to topological sort’s DFS approach).
2. Build the transposed graph (every edge reversed).
3. Pop the stack one node at a time; for every still-unvisited node, run DFS on the **transposed** graph – everything reached in this one DFS call is exactly one SCC.

### C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void fillOrder(int node, vector<vector<int>>& adj, vector<bool>& visited, stack<int>
5   & order) {
6     visited[node] = true;
7     for (int neighbor : adj[node]) {
8         if (!visited[neighbor]) fillOrder(neighbor, adj, visited, order);
9     }
10    order.push(node);
11 }
12
13 void collectSCC(int node, vector<vector<int>>& transposed, vector<bool>& visited,
14   vector<int>& component) {
15     visited[node] = true;
16     component.push_back(node);
17     for (int neighbor : transposed[node]) {

```

```

17         if (!visited[neighbor]) collectSCC(neighbor, transposed, visited, component
18             );
19     }
20 }
21 vector<vector<int>> kosaraju(int n, vector<vector<int>>& adj) {
22     vector<bool> visited(n, false);
23     stack<int> order;
24     for (int i = 0; i < n; i++) {
25         if (!visited[i]) fillOrder(i, adj, visited, order);
26     }
27
28     vector<vector<int>> transposed(n);
29     for (int u = 0; u < n; u++) {
30         for (int v : adj[u]) transposed[v].push_back(u);
31     }
32
33     fill(visited.begin(), visited.end(), false);
34     vector<vector<int>> sccs;
35     while (!order.empty()) {
36         int node = order.top();
37         order.pop();
38         if (!visited[node]) {
39             vector<int> component;
40             collectSCC(node, transposed, visited, component);
41             sccs.push_back(component);
42         }
43     }
44     return sccs;
45 }
46
47 int main() {
48     int n = 5;
49     vector<vector<int>> adj(n);
50     adj[0] = {2}; adj[2] = {1}; adj[1] = {0}; adj[0].push_back(3); adj[3] = {4};
51
52     for (auto& scc : kosaraju(n, adj)) {
53         for (int x : scc) cout << x << " ";
54         cout << endl;
55     }
56     // 0 1 2    (one SCC)
57     // 3        (its own SCC)
58     // 4        (its own SCC)
59     return 0;
60 }

```

## Dry run

**Setup.** Take edges  $0 \rightarrow 2, 2 \rightarrow 1, 1 \rightarrow 0, 0 \rightarrow 3, 3 \rightarrow 4$ .

First DFS (finish-time order, starting from 0): visits  $0 \rightarrow 2 \rightarrow 1$  (back to 0, already visiting, skip), 1 finishes first (push), 2 finishes (push), then 0 continues to  $3 \rightarrow 4$ , 4 finishes (push), 3 finishes (push), 0 finishes (push). Stack (bottom-to-top): 1, 2, 4, 3, 0. Transposed graph:  $2 \rightarrow 0, 1 \rightarrow 2, 0 \rightarrow 1, 3 \rightarrow 0, 4 \rightarrow 3$ . Second DFS, popping stack top-to-bottom: pop 0 first – DFS on transposed graph from 0:  $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$  (visited, stop) – component  $\{0, 1, 2\}$ . Pop 3 (next unvisited): DFS from 3 on transposed graph:  $3 \rightarrow 0$  (visited, stop) – component  $\{3\}$ . Pop 4: component  $\{4\}$ . Three SCCs:  $\{0, 1, 2\}, \{3\}, \{4\}$  – correctly identifying the 3-cycle as one SCC and the two remaining nodes (no return path) as singleton SCCs.

**Complexity**

**Time Complexity.**  $O(V + E)$  – two full DFS passes plus building the transpose, each linear in the graph size. **Space Complexity.**  $O(V + E)$  for the transposed adjacency list, plus  $O(V)$  for the stack and visited arrays.

**Takeaway**

Kosaraju’s algorithm is this book’s clearest demonstration that **reversing** a directed graph’s edges is sometimes itself the key algorithmic move – not just bookkeeping, but the step that turns an otherwise hard global structure question into two simple, linear DFS passes.

## 1.3 Bridges in a Graph

**Problem Statement.** Given an undirected, connected graph, find every **bridge** – an edge whose removal would increase the number of connected components (disconnect the graph).

**Pattern Recognition**

“Find edges whose removal disconnects the graph” is solved by a single DFS tracking *tin* and *low* (Section 1.1): a tree edge  $u \rightarrow v$  (where  $v$  is  $u$ ’s child in the DFS tree) is a bridge exactly when  $low[v] > tin[u]$  – meaning  $v$ ’s entire subtree has **no** back edge reaching  $u$  or any of  $u$ ’s ancestors, so the only way to reach  $v$ ’s subtree from the rest of the graph is through this one edge.

### Deriving the *low*-value update rule

While DFS explores from node  $u$ , for each neighbor  $v$ :

- If  $v$  is  $u$ ’s **parent** (the edge just arrived through), skip it entirely – this is not a genuine alternative path.
- If  $v$  is **unvisited**, it becomes a tree-edge child: recurse, then update  $low[u] \leftarrow \min(low[u], low[v])$  (whatever  $v$ ’s subtree can reach back to,  $u$  can also reach back to, through  $v$ ). Check the bridge condition here:  $low[v] > tin[u]$ .
- If  $v$  is **already visited** (and not the parent), this is a back edge: update  $low[u] \leftarrow \min(low[u], tin[v])$  – using  $v$ ’s **discovery time** directly (not  $low[v]$ , since crossing a back edge to  $v$  only guarantees reaching  $v$  itself, not necessarily anything  $v$ ’s own low value might represent through edges not yet explored from  $u$ ’s perspective).

### C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int timer = 0;
5
6 void dfs(int node, int parent, vector<vector<int>>& adj, vector<bool>& visited,
7         vector<int>& tin, vector<int>& low, vector<vector<int>>& bridges) {
8     visited[node] = true;
9     tin[node] = low[node] = timer++;

```

```

10
11     for (int neighbor : adj[node]) {
12         if (neighbor == parent) continue;
13
14         if (visited[neighbor]) {
15             low[node] = min(low[node], tin[neighbor]); // back edge
16         } else {
17             dfs(neighbor, node, adj, visited, tin, low, bridges);
18             low[node] = min(low[node], low[neighbor]);
19             if (low[neighbor] > tin[node]) {
20                 bridges.push_back({node, neighbor});
21             }
22         }
23     }
24 }
25
26 vector<vector<int>> findBridges(int n, vector<vector<int>>& adj) {
27     vector<bool> visited(n, false);
28     vector<int> tin(n), low(n);
29     vector<vector<int>> bridges;
30     timer = 0;
31
32     for (int i = 0; i < n; i++) {
33         if (!visited[i]) dfs(i, -1, adj, visited, tin, low, bridges);
34     }
35     return bridges;
36 }
37
38 int main() {
39     int n = 4;
40     vector<vector<int>> adj(n);
41     auto addEdge = [&](int u, int v) { adj[u].push_back(v); adj[v].push_back(u); };
42     addEdge(0, 1); addEdge(1, 2); addEdge(2, 0); addEdge(1, 3);
43
44     for (auto& b : findBridges(n, adj)) cout << b[0] << "-" << b[1] << " ";
45     cout << endl; // 1-3
46     return 0;
47 }

```

## Dry run

**Setup.** Take edges 0–1, 1–2, 2–0, 1–3 (a triangle 0, 1, 2 with a pendant edge to 3), DFS from 0.

$dfs(0, -1)$ :  $tin[0] = low[0] = 0$ . Neighbor 1: unvisited, recurse.  $dfs(1, 0)$ :  $tin[1] = low[1] = 1$ . Neighbor 0: is parent, skip. Neighbor 2: unvisited, recurse.  $dfs(2, 1)$ :  $tin[2] = low[2] = 2$ . Neighbor 1: is parent, skip. Neighbor 0: already visited, not parent – back edge,  $low[2] = \min(2, tin[0]=0) = 0$ . Return to 1:  $low[1] = \min(1, low[2]=0) = 0$ ; check bridge:  $low[2]=0 > tin[1]=1$ ? No – not a bridge (correct: edge 1–2 sits on the triangle’s cycle). Back at 1, next neighbor 3: unvisited, recurse.  $dfs(3, 1)$ :  $tin[3] = low[3] = 3$ ; no unvisited or back-edge neighbors. Return to 1:  $low[1] = \min(0, low[3]=3) = 0$ ; check bridge:  $low[3]=3 > tin[1]=1$ ? **Yes** – edge 1–3 is a bridge (correct: it’s the only connection to the otherwise-isolated node 3). Return to 0:  $low[0] = \min(0, low[1]=0) = 0$ ; check bridge:  $low[1]=0 > tin[0]=0$ ? No – not a bridge.

## Complexity

**Time Complexity.**  $O(V + E)$  – a single DFS pass. **Space Complexity.**  $O(V)$  for  $tin$ ,  $low$ ,  $visited$ , and the recursion stack.

**Takeaway**

The distinction between updating *low* with a child's *low* value (tree edges) versus a neighbor's *tin* value (back edges) is the single most important detail to get right in this algorithm – using *low* for a back edge would incorrectly let information “leak” through a node that hasn't finished exploring its own subtree yet.

## 1.4 Articulation Points

**Problem Statement.** Given an undirected, connected graph, find every **articulation point** (cut vertex) – a node whose removal (along with all its incident edges) would increase the number of connected components.

**Pattern Recognition**

This closing problem of the entire graph series reuses Section 1.3's exact *tin/low* DFS, with the condition shifted from edges to **nodes**, and from strict to non-strict inequality: a non-root node *u* is an articulation point if it has a child *v* with  $low[v] \geq tin[u]$  – plus one special case for the DFS **root**, which has no parent for the usual argument to apply to.

### Why $\geq$ replaces bridges' strict $>$

A bridge requires *v*'s subtree to have **no** way back to *u* or higher ( $low[v] > tin[u]$ , strictly). An articulation point is a weaker condition: *u* is critical even if *v*'s subtree can reach back *exactly* to *u* itself ( $low[v] = tin[u]$ ) – because reaching back only as far as *u* still means removing *u* disconnects *v*'s subtree from everything *above u*, even though *v*'s subtree remains internally connected to *u*'s position. Only if *v*'s subtree can reach back *above u* (to some ancestor,  $low[v] < tin[u]$ ) does removing *u* fail to disconnect anything.

### The root's special case

The root of the DFS tree has no parent, so the “can my child's subtree reach above me” question doesn't apply to it the same way. Instead: the root is an articulation point exactly when it has **more than one child** in the DFS tree – each child starts an independent branch that (by definition of being a separate DFS tree child rather than reached via a back edge) has no other connection to the rest of the graph except through the root.

### C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int timer = 0;
5
6 void dfs(int node, int parent, vector<vector<int>>& adj, vector<bool>& visited,
7         vector<int>& tin, vector<int>& low, vector<bool>& isArticulation) {
8     visited[node] = true;
9     tin[node] = low[node] = timer++;
10    int children = 0;
11
12    for (int neighbor : adj[node]) {
13        if (neighbor == parent) continue;

```

```

14         if (visited[neighbor]) {
15             low[node] = min(low[node], tin[neighbor]);
16         } else {
17             dfs(neighbor, node, adj, visited, tin, low, isArticulation);
18             low[node] = min(low[node], low[neighbor]);
19
20             if (parent != -1 && low[neighbor] >= tin[node]) {
21                 isArticulation[node] = true;
22             }
23             children++;
24         }
25     }
26 }
27
28 if (parent == -1 && children > 1) {
29     isArticulation[node] = true;
30 }
31 }
32
33 vector<int> findArticulationPoints(int n, vector<vector<int>>& adj) {
34     vector<bool> visited(n, false), isArticulation(n, false);
35     vector<int> tin(n), low(n);
36     timer = 0;
37
38     for (int i = 0; i < n; i++) {
39         if (!visited[i]) dfs(i, -1, adj, visited, tin, low, isArticulation);
40     }
41
42     vector<int> result;
43     for (int i = 0; i < n; i++) {
44         if (isArticulation[i]) result.push_back(i);
45     }
46     return result;
47 }
48
49 int main() {
50     int n = 5;
51     vector<vector<int>> adj(n);
52     auto addEdge = [&](int u, int v) { adj[u].push_back(v); adj[v].push_back(u); };
53     addEdge(0, 1); addEdge(1, 2); addEdge(2, 0); addEdge(1, 3); addEdge(3, 4);
54
55     for (int x : findArticulationPoints(n, adj)) cout << x << " ";
56     cout << endl; // 1 3
57     return 0;
58 }

```

## Dry run

**Setup.** Take edges 0–1, 1–2, 2–0, 1–3, 3–4 (a triangle 0, 1, 2, with 1 also connected via 3 to a pendant 4), DFS from 0.

This is the same graph shape as Section 1.3's example with an added pendant (3–4). Following the same DFS order: *tin/low* for 0, 1, 2 resolve identically (the triangle is unaffected). At node 3 (child of 1, with one child of its own, 4): *dfs*(4, 3) gives *tin*[4] = *low*[4] = 4. Back at 3: *low*[3] = min(3, 4) = 3; check articulation: parent of 3 is 1 ( $\neq -1$ ), *low*[4]=4  $\geq$  *tin*[3]=3? Yes – 3 **is an articulation point** (removing it disconnects 4). Back at 1 (processing child 3): *low*[1] = min(0, *low*[3]=3) = 0; check articulation: parent of 1 is 0 ( $\neq -1$ ), *low*[3]=3  $\geq$  *tin*[1]=1? Yes – 1 **is an articulation point** (removing it disconnects {3, 4} from the triangle). Node 0 (the root): only one DFS-tree child (1; node 2 was reached via a back edge, not as a tree child), so the root special case does **not** apply – 0 is not an articulation point. Final: {1, 3}.

**Complexity**

**Time Complexity.**  $O(V + E)$  – a single DFS pass, identical structure to bridge-finding.

**Space Complexity.**  $O(V)$  for *tin*, *low*, *visited*, *isArticulation*, and the recursion stack.

**Takeaway**

This book series closes its graph coverage on a fitting note: bridges and articulation points are, almost letter for letter, the same algorithm, distinguished only by a strict-versus-non-strict inequality and one root-specific special case. Across every chapter in this series – Greedy, Binary Search, Heap, Stack/Queue, BST, Trie, Recursion, eight DP sub-topics, and six graph sub-topics – the recurring lesson has been the same: a small number of genuinely new ideas, recognized and reapplied with small, deliberate modifications, cover an enormous space of distinct-looking interview problems.